Practical 5

Implementation of knapsack problem using dynamic programming

1. Introduction

The Knapsack Problem is a classic optimization problem in computer science and algorithm design. In the 0/1 Knapsack Problem, we are given a set of items, where each item has a weight and a value. We need to select items such that the total weight does not exceed the given capacity of the knapsack while maximizing the total value.

The term 0/1 means that each item can either be selected completely (1) or not selected (0). Dynamic Programming is used to efficiently solve this problem by breaking it into smaller sub problems and storing previously calculated results to avoid repeated computations.

2. Aim

To implement the 0/1 Knapsack Problem using Dynamic Programming in Python and determine the maximum value that can be obtained without exceeding the given knapsack capacity.

3. Objectives

- To understand the concept of the 0/1 Knapsack Problem.
- To understand the Dynamic Programming approach.
- To find the optimal selection of items.
- To maximize the total value within the given capacity.
- To analyze the time and space complexity of the algorithm. □ To implement the algorithm using Python.

4. Algorithm

Step 1: Start.

Step 2: Read the number of items, their weights, their values, and the knapsack capacity.

Step 3: Create a DP table dp[i][w], where:

- i represents the number of items considered.
- w represents the current capacity.

Step 4: Initialize all values in the DP table to 0.

Step 5: For each item, check whether its weight is less than or equal to the current capacity.

Step 6: If the item can be included, calculate:

valuei +dp[i−1][w−weighti] and

compare it with excluding the item:

dp[i−1][w]

Step 7: Store the maximum of these two values in the DP table.

Step 8: If the item's weight is greater than the current capacity, exclude the item.

Step 9: The value stored at dp[n][capacity] is the maximum possible value.

Step 10: Stop.

5. Pseudocode

```
BEGIN

FUNCTION Knapsack(weights, values, capacity)

   n ← number of items

   Create DP table dp[n+1][capacity+1]

   Initialize all elements of dp to 0

   FOR i ← 1 TO n
      FOR w ← 1 TO capacity

         IF weights[i-1] ≤ w THEN
```

```
            include ← values[i-1] + dp[i-
            1][w - weights[i-1]] exclude ←
            dp[i-1][w]

            dp[i][w] ← maximum(include, exclude)
        ELSE dp[i][w] ← dp[i-
            1][w]
        END IF
    END FOR
  END FOR

  RETURN dp[n][capacity]

END FUNCTION

END
```

Code:

```python
def knapsack(weights, values, capacity):
    n = len(values)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][capacity]
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 5
maximum_value = knapsack(weights, values, capacity)

print("Weights:", weights)
print("Values:", values)
print("Knapsack Capacity:", capacity)
print("Maximum Value:", maximum_value)
```

```
Weights: [2, 3, 4, 5]
Values: [3, 4, 5, 6]
Knapsack Capacity: 5
Maximum Value: 7
```

6. Advantages

- Provides an optimal solution to the 0/1 Knapsack Problem.
- Avoids repeated calculations by storing previous results.
- Easier to understand and implement compared to some advanced optimization techniques.
- Suitable for problems where the capacity is relatively small. □ Has polynomial time complexity of O(n × W).

7. Disadvantages

- Requires additional memory to store the DP table.
- Space complexity is O(n × W).
- Can become inefficient when the knapsack capacity is very large.
- It is specifically designed for the 0/1 Knapsack Problem and does not directly solve the fractional version.

- The DP table can consume significant memory for large inputs.

8. Complexity Analysis

Complexity	Value
Time Complexity	O(n × W)
Space Complexity	O(n × W)

Where:

- n = number of items
- W = capacity of the knapsack

9. Conclusion

The 0/1 Knapsack Problem using Dynamic Programming was successfully implemented in Python. The algorithm considers each item either selected or not selected and stores the results of smaller subproblems in a DP table. This avoids unnecessary repeated calculations and produces the maximum possible value within the given capacity. Thus, Dynamic Programming provides an efficient and systematic approach for solving the 0/1 Knapsack optimization problem.